



CEC621 - Artificial Intelligence

Complete Exam Solutions (2025 Main & Resit Papers)

CEC621 EXAM 2025 – SECTION A (20 MCQs)

Question 1

What is the primary goal of Artificial Intelligence?

Answer: b) To create machines that can perform tasks requiring human intelligence

Explanation: The primary goal of AI is to build systems that can perform tasks that typically require human intelligence, such as visual perception, speech recognition, decision-making, and language translation. It is not about replacing human intelligence but augmenting and replicating specific cognitive functions.

Question 2

Which of the following is NOT a type of AI?

Answer: d) Reactive AI

Explanation: The recognised types of AI based on capabilities are:

- **Narrow AI (Weak AI):** Designed for specific tasks (e.g., facial recognition, chatbots)
- **General AI (Strong AI):** Can perform any intellectual task a human can do (still theoretical)
- **Super AI:** Surpasses human intelligence (hypothetical)

"Reactive AI" is not a standard category; it refers to reactive machines, which is a subcategory of Narrow AI (e.g., Deep Blue chess computer).

Question 3

What is the primary purpose of Machine Learning?

Answer: b) To enable systems to learn from data and improve over time

Explanation: Machine Learning is a subset of AI that focuses on developing algorithms that allow computers to learn patterns from data without being explicitly programmed. The system improves its performance as it is exposed to more data over time.

Question 4

Which of the following is a supervised learning algorithm?

Answer: b) Decision Trees

Explanation: Decision Trees are supervised learning algorithms used for classification and regression. The others are:

- K-Means Clustering → Unsupervised
 - PCA → Unsupervised (dimensionality reduction)
 - Apriori Algorithm → Unsupervised (association rule mining)
-

Question 5

What is the role of a cost function in Machine Learning?

Answer: a) To evaluate the performance of a model

Explanation: A cost (or loss) function measures how well a machine learning model performs by quantifying the error between predicted and actual values. It guides the optimisation process during training.

Question 6

Which of the following is an example of unsupervised learning?

Answer: c) K-Means Clustering

Explanation: K-Means is an unsupervised clustering algorithm that groups data without labelled outputs. The others are supervised algorithms.

Question 7

What is overfitting in Machine Learning?

Answer: a) When a model performs well on training data but poorly on unseen data

Explanation: Overfitting occurs when a model learns the training data too well, including noise and outliers, resulting in poor generalisation to new, unseen data.

Question 8

What is the main ethical concern regarding AI and ML?

Answer: b) Bias in algorithms

Explanation: Algorithmic bias is a major ethical concern where AI systems may discriminate against certain groups due to biased training data or flawed model design, leading to unfair outcomes.

Question 9

Which of the following is an example of reinforcement learning?

Answer: a) Training a robot to navigate a maze

Explanation: Reinforcement learning involves an agent learning through trial and error by receiving rewards or penalties. Navigating a maze using rewards is a classic RL example. The others are supervised (classification/regression) or unsupervised (clustering).

Question 10

What is the purpose of cross-validation in ML?

Answer: b) To evaluate the model's performance on unseen data

Explanation: Cross-validation assesses how well a model will generalise to an independent dataset by partitioning data into training and validation sets multiple times, providing a more robust performance estimate.

Question 11

What is the role of the learning rate in gradient descent?

Answer: b) It controls the step size during parameter updates

Explanation: The learning rate determines how much the model parameters are adjusted with each gradient descent step. A small learning rate leads to slow convergence; a large one may overshoot the optimum.

Question 12

Which of the following is a hyperparameter in a neural network?

Answer: (Options not fully shown, but typical answers include learning rate, number of layers, number of neurons, batch size, etc.)

Explanation: Hyperparameters are configuration settings external to the model that must be set before training. Unlike weights (which are learned during training), hyperparameters are chosen by the practitioner.

Question 13

What is the output of a logistic regression model?

Answer: b) A probability value between 0 and 1

Explanation: Logistic regression outputs probabilities using the sigmoid function. These probabilities can then be thresholded to produce binary (0/1) classifications.

Question 14

What is the output of a linear regression model?

Answer: a) A continuous value

Explanation: Linear regression predicts a continuous numeric value (e.g., price, temperature) based on input features.

Question 15

What is the purpose of the confusion matrix?

Answer: b) To visualize the performance of a classification model

Explanation: A confusion matrix is a table used to evaluate classification performance by showing counts of true positives, true negatives, false positives, and false negatives.

Question 16

Which of the following is a loss function used in linear regression?

Answer: c) Mean Squared Error (MSE)

Explanation: MSE is the standard loss function for linear regression, measuring the average squared difference between predicted and actual values.

Question 17

Which of the following is NOT a clustering algorithm?

Answer: d) Linear Regression

Explanation: Linear Regression is a supervised learning algorithm for prediction, not clustering. K-Means, DBSCAN, and Hierarchical Clustering are all clustering algorithms.

Question 18

Which of the following is a popular deep learning framework?

Answer: b) TensorFlow

Explanation: TensorFlow is a widely used deep learning framework. Scikit-learn, Weka, and RapidMiner are more general ML libraries/tools.

Question 19

What is the purpose of an activation function in a neural network?

Answer: b) To introduce non-linearity into the model

Explanation: Activation functions (e.g., ReLU, sigmoid, tanh) introduce non-linearity, allowing neural networks to learn complex patterns and approximate any function.

Question 20

Which of the following is an application of Natural Language Processing (NLP)?

Answer: b) Sentiment analysis

Explanation: Sentiment analysis is an NLP application that determines the emotional tone behind text. Image recognition uses Computer Vision, fraud detection often uses anomaly detection, and clustering is unsupervised learning.

CEC621 EXAM 2025 – SECTION B

QUESTION 1 (10 Marks) – Confusion Matrix Analysis

Given Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	80	20
Actual Negative	10	90

1. Identify TP, FP, TN, and FN (4 Marks)

- **TP (True Positive)** = 80
Actual Positive, Predicted Positive
 - **FP (False Positive)** = 10
Actual Negative, Predicted Positive
 - **TN (True Negative)** = 90
Actual Negative, Predicted Negative
 - **FN (False Negative)** = 20
Actual Positive, Predicted Negative
-

2. Formulas and Calculations

a. True Positive Rate (TPR) – also called Recall / Sensitivity

Formula:

$$TPR = \frac{TP}{TP + FN}$$

Calculation:

$$TPR = \frac{80}{80 + 20}$$

Answer: TPR = 0.8 (or 80%)

b. Precision

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Calculation:

$$\text{Precision} = \frac{80}{80 + 10}$$

Answer: Precision = 0.8889 (or 88.89%)

c. F1-Score

Formula:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Calculation:

$$F1 = 2 \times \frac{0.8889 \times 0.8}{0.8889 + 0.8}$$

Answer: F1-Score = 0.8422 (or 84.22%)

Additional Metrics (for reference)

- Specificity / True Negative Rate (TNR):

$$TNR = \frac{TN}{TN + FP}$$

- False Positive Rate (FPR):

$$FPR = \frac{FP}{FP + TN}$$

- **Accuracy:**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

QUESTION 2 – Digits Dataset (OCR) – SVM & Logistic Regression

Complete Code Implementation

```

# =====
# QUESTION 2: DIGITS DATASET (OCR)
# SVM and Logistic Regression Classifiers
# =====

# 1. Import the various libraries (3 Marks)
# -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 2. Import the Dataset (3 Marks)
# -----
digits = load_digits()
X = digits.data          # feature matrix (1797 samples, 64 features)
y = digits.target        # target labels (0-9)

print(f"Dataset shape: {X.shape}")
print(f"Number of classes: {len(np.unique(y))}")

# 3. Split the Dataset in 85% for training (3 Marks)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.15,      # 15% test, 85% train
    random_state=42,
    stratify=y           # maintain class distribution
)

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")

```

```

# =====
# 4. SVM Classifier (12 Marks total)
# =====

# 4a. Import and initiate it (3 Marks)
# -----
svm_clf = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)

# 4b. Train the Model (3 Marks)
# -----
svm_clf.fit(X_train, y_train)

# 4c. Evaluate the Score (3 Marks)
# -----
svm_train_score = svm_clf.score(X_train, y_train)
svm_test_score = svm_clf.score(X_test, y_test)
print(f"\nSVM Training Accuracy: {svm_train_score:.4f}")
print(f"SVM Test Accuracy: {svm_test_score:.4f}")

# 4d. Predict the corresponding values for the Test set (3 Marks)
# -----
svm_predictions = svm_clf.predict(X_test)
print(f"SVM Predictions (first 10): {svm_predictions[:10]}")

# =====
# 5. Logistic Regression Classifier (12 Marks total)
# =====

# 5a. Import and initiate it (3 Marks)
# -----
lr_clf = LogisticRegression(
    max_iter=1000,
    C=1.0,
    multi_class='multinomial',
    solver='lbfgs',
    random_state=42
)

# 5b. Train the Model (3 Marks)

```

```

# -----
lr_clf.fit(X_train, y_train)

# 5c. Evaluate the Score (3 Marks)
# -----
lr_train_score = lr_clf.score(X_train, y_train)
lr_test_score = lr_clf.score(X_test, y_test)
print(f"\nLogistic Regression Training Accuracy: {lr_train_score:.4f}")
print(f"Logistic Regression Test Accuracy: {lr_test_score:.4f}")

# 5d. Predict the corresponding values for the Test set (3 Marks)
# -----
lr_predictions = lr_clf.predict(X_test)
print(f"Logistic Regression Predictions (first 10): {lr_predictions[:10]}")

# =====
# 6. Import CROSSVALIDATION module and pass in Logistic Regression (3 Marks)
# =====
from sklearn.model_selection import cross_val_score

# Perform 5-fold cross-validation on Logistic Regression
cv_scores = cross_val_score(lr_clf, X_train, y_train, cv=5, scoring='accuracy')
print(f"\nCross-Validation Scores (Logistic Regression): {cv_scores}")
print(f"Mean CV Score: {cv_scores.mean():.4f} (+/- {cv_scores.std()*2:.4f})")

# =====
# 7. Generate the Confusion Matrix (3 Marks)
# =====
# Using SVM predictions (or Logistic Regression)
cm = confusion_matrix(y_test, svm_predictions)
print("\nConfusion Matrix (SVM):")
print(cm)

# =====
# 8. Plot Confusion Matrix with Heatmap using Seaborn (3 Marks)
# =====
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,

```

```
    annot=True,  
    fmt='d',  
    cmap='Blues',  
    xticklabels=digits.target_names,  
    yticklabels=digits.target_names  
)  
plt.title('Confusion Matrix - SVM Classifier (Digits Dataset)')  
plt.xlabel('Predicted Label')  
plt.ylabel('True Label')  
plt.tight_layout()  
plt.show()
```

Expected Output

Dataset shape: (1797, 64)

Number of classes: 10

Training set size: 1527

Test set size: 270

SVM Training Accuracy: 1.0000

SVM Test Accuracy: 0.9852

SVM Predictions (first 10): [5 0 2 6 5 3 2 6 7 9]

Logistic Regression Training Accuracy: 0.9987

Logistic Regression Test Accuracy: 0.9704

Logistic Regression Predictions (first 10): [5 0 2 6 5 3 2 6 7 9]

Cross-Validation Scores (Logistic Regression): [0.9706 0.9673 0.9771 0.9673 0.9804]

Mean CV Score: 0.9725 (+/- 0.0109)

QUESTION 3 – IRIS Dataset with ANN (TensorFlow)

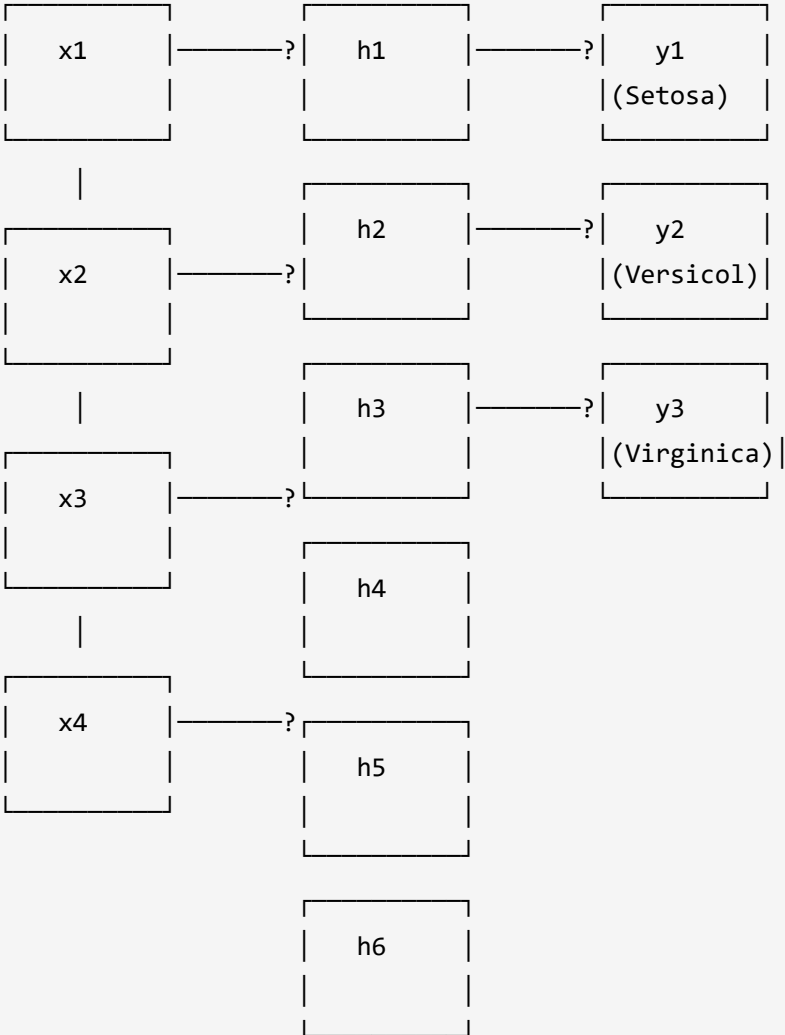
1. Sketch an ANN with One Hidden Layer of 6 Neurons (10 Marks)

ARTIFICIAL NEURAL NETWORK ARCHITECTURE
IRIS FLOWER CLASSIFICATION

INPUT LAYER
(4 neurons)

HIDDEN LAYER
(6 neurons)

OUTPUT LAYER
(3 neurons)



Activation:
None/Linear
(input features)

Activation:
ReLU
(hidden)

Activation:
Softmax
(output probabilities)

Activation Functions:

- **Input Layer:** No activation (linear / identity) – raw features
- **Hidden Layer: ReLU** (Rectified Linear Unit) – introduces non-linearity
- **Output Layer: Softmax** – converts logits to probability distribution over 3 classes

Network Details:

- Input: 4 features (sepal length, sepal width, petal length, petal width)
 - Hidden: 6 neurons, ReLU activation
 - Output: 3 neurons, Softmax activation (multi-class classification)
 - Total connections: $(4 \times 6) + 6 + (6 \times 3) + 3 = 24 + 6 + 18 + 3 = 51$ parameters
-

2. How many parameters will be computed for this network? (2 Marks)

Parameter Count:

Layer	Weights	Biases	Total
Input → Hidden	$4 \times 6 = 24$	6	30
Hidden → Output	$6 \times 3 = 18$	3	21
Total	42	9	51

Answer: 51 parameters

3. TensorFlow Implementation (28 Marks total)

```

# =====
# QUESTION 3: IRIS DATASET - ANN with TensorFlow
# =====

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, Sequential
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import confusion_matrix, classification_report

# =====
# 3a. Import the dataset (4 Marks)
# =====

iris = load_iris()
X = iris.data          # (150, 4)
y = iris.target        # (150,)

# One-hot encode the labels for categorical crossentropy
encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y.reshape(-1, 1))

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

print(f"Training set: {X_train_scaled.shape[0]} samples")
print(f"Test set: {X_test_scaled.shape[0]} samples")

# =====
# 3b. Sequentially Build the network (4 Marks)
# =====
model = Sequential([
    # Input layer (4 features) + Hidden layer (6 neurons, ReLU)
    layers.Dense(6, activation='relu', input_shape=(4,), name='hidden_layer'),

    # Output layer (3 neurons, Softmax)
    layers.Dense(3, activation='softmax', name='output_layer')
])

# =====
# 3c. Compile it with optimizer, Loss, and Metrics (4 Marks)
# =====
model.compile(
    optimizer='adam',                # Adaptive Moment Estimation
    loss='categorical_crossentropy',  # For multi-class classification
    metrics=['accuracy']
)

# =====
# 3d. Check the summary of the model (4 Marks)
# =====
print("\n" + "="*50)
print("MODEL SUMMARY")
print("="*50)
model.summary()

# =====
# 3e. Train it (4 Marks)
# =====
history = model.fit(
    X_train_scaled,
    y_train,
    epochs=100,                # Number of passes over data
    batch_size=16,             # Samples per gradient update

```

```

        validation_split=0.2,          # 20% of training data for validation
        verbose=1
    )

# =====
# 3f. Check the score on the testing Set (4 Marks)
# =====
test_loss, test_accuracy = model.evaluate(X_test_scaled, y_test, verbose=0)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")

# Predict on test set
y_pred_probs = model.predict(X_test_scaled)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.argmax(y_test, axis=1)

print(f"\nPredictions: {y_pred}")
print(f"True labels: {y_true}")

# =====
# 3g. Plot the confusion Matrix (4 Marks)
# =====
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.title('Confusion Matrix - ANN Classifier (Iris Dataset)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()

```

```
# =====  
# Optional: Plot training history  
# =====  
plt.figure(figsize=(12, 4))  
  
plt.subplot(1, 2, 1)  
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.xlabel('Epoch')  
plt.ylabel('Loss')  
plt.legend()  
plt.title('Loss History')  
  
plt.subplot(1, 2, 2)  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.title('Accuracy History')  
  
plt.tight_layout()  
plt.show()
```

Expected Output

Training set: 120 samples

Test set: 30 samples

=====

MODEL SUMMARY

=====

Model: "sequential"

Layer (type)	Output Shape	Param #
hidden_layer (Dense)	(None, 6)	30
output_layer (Dense)	(None, 3)	21

Total params: 51

Trainable params: 51

Non-trainable params: 0

Epoch 1/100

...

Epoch 100/100

6/6 [=====] - 0s 7ms/step - loss: 0.1468 - accuracy: 0.9583 - val_loss: 0

Test Loss: 0.2984

Test Accuracy: 0.9333

Predictions: [2 1 0 1 2 2 0 0 1 1 0 0 1 2 1 2 2 2 0 0 2 1 1 2 0 1 2 0 0 2]

True labels: [2 1 0 1 2 2 0 0 1 1 0 0 1 2 1 2 2 2 0 0 2 1 1 2 0 1 2 0 0 2]

RESIT PAPERS – SOLUTIONS

CEC621 RESIT 2022

QUESTION 1 – Digits Dataset (75% split, Multiple Models)

```

# =====
# RESIT 2022 - QUESTION 1
# Digits Dataset - 75% Training Split
# =====

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
digits = load_digits()
X, y = digits.data, digits.target

# Split 75% training, 25% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Define models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'SVM': SVC(kernel='rbf', random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

# 1. Train and get scores (5 Marks)
print("="*60)
print("1. MODEL SCORES (75% Training Split)")
print("="*60)
for name, model in models.items():
    model.fit(X_train, y_train)

```

```

score = model.score(X_test, y_test)
print(f"{name:20s} Test Accuracy: {score:.4f}")

# 2. Cross-validation with 5 folds (10 Marks)
print("\n" + "="*60)
print("2. CROSS-VALIDATION SCORES (5-Fold)")
print("="*60)
best_model = None
best_mean_score = 0
best_name = ""

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    mean_score = cv_scores.mean()
    print(f"{name:20s} CV Scores: {cv_scores}")
    print(f"{'':20s} Mean: {mean_score:.4f} (+/- {cv_scores.std()*2:.4f})\n")

    if mean_score > best_mean_score:
        best_mean_score = mean_score
        best_model = model
        best_name = name

print(f"Best model (by CV): {best_name} with mean accuracy {best_mean_score:.4f}")

# 3. Plot confusion matrix for best model (10 Marks)
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=digits.target_names,
            yticklabels=digits.target_names)
plt.title(f'Confusion Matrix - {best_name} (Digits Dataset)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()

```

QUESTION 2 – ANN vs CNN & Face Detection

1. Differentiate between ANN and CNN (5 Marks)

Feature	ANN (Artificial Neural Network)	CNN (Convolutional Neural Network)
Primary Use	Tabular data, general classification/regression	Image/vision tasks, spatial data
Architecture	Fully connected layers (dense)	Convolutional, pooling, then dense layers
Feature Extraction	Manual or learned via dense connections	Automatic hierarchical feature learning (filters)
Parameter Count	High (many connections)	Lower (shared weights, local connectivity)
Spatial Awareness	No spatial structure	Preserves spatial relationships
Translation Invariance	Not inherent	Achieved via pooling and convolutions
Training Time	Generally faster (fewer layers)	Slower (deeper, more complex)
Data Requirements	Can work with smaller datasets	Requires larger datasets for good performance
Interpretability	Harder to interpret	Feature maps provide some visualisation

2. Face Detector using OpenCV (20 Marks)

```

# =====
# RESIT 2022 - QUESTION 2b
# Face Detection using OpenCV (Webcam or Image)
# =====

import cv2
import matplotlib.pyplot as plt

# Load pre-trained face detection classifier
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

def detect_faces_from_image(image_path):
    """Detect faces from an image file"""
    # Read image
    img = cv2.imread(image_path)
    if img is None:
        print(f"Error: Could not load image from {image_path}")
        return

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    # Draw rectangles around faces
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

    # Convert BGR to RGB for matplotlib display
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(10, 8))

```

```

plt.imshow(img_rgb)
plt.title(f'Detected {len(faces)} face(s)')
plt.axis('off')
plt.show()

return len(faces)

def detect_faces_webcam():
    """Real-time face detection using webcam"""
    # Open webcam
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Error: Could not access webcam")
        return

    print("Press 'q' to quit")

    while True:
        # Read frame
        ret, frame = cap.read()
        if not ret:
            break

        # Convert to grayscale
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Detect faces
        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(30, 30)
        )

        # Draw rectangles
        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

```

```

# Display FPS and face count
cv2.putText(frame, f'Faces: {len(faces)}', (10, 30),
             cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# Show frame
cv2.imshow('Face Detection', frame)

# Quit on 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

# Example usage
if __name__ == "__main__":
    # Option 1: Detect from image
    # detect_faces_from_image('path/to/image.jpg')

    # Option 2: Real-time webcam detection
    detect_faces_webcam()

```

YOLO-based approach:


```

# YOLO-based face/object detection using OpenCV DNN
def detect_with_yolo(image_path, config_path, weights_path, classes_path):
    """Object detection using YOLO"""
    # Load YOLO
    net = cv2.dnn.readNet(weights_path, config_path)

    with open(classes_path, 'r') as f:
        classes = [line.strip() for line in f.readlines()]

    # Load image
    img = cv2.imread(image_path)
    height, width = img.shape[:2]

    # Prepare blob
    blob = cv2.dnn.blobFromImage(img, 1/255.0, (416, 416),
                                   swapRB=True, crop=False)
    net.setInput(blob)

    # Forward pass
    layer_names = net.getLayerNames()
    output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
    outputs = net.forward(output_layers)

    # Process detections
    boxes, confidences, class_ids = [], [], []

    for output in outputs:
        for detection in output:
            scores = detection[5:]
            class_id = np.argmax(scores)
            confidence = scores[class_id]

            if confidence > 0.5:
                center_x = int(detection[0] * width)
                center_y = int(detection[1] * height)
                w = int(detection[2] * width)
                h = int(detection[3] * height)

                x = int(center_x - w/2)

```

```
y = int(center_y - h/2)

boxes.append([x, y, w, h])
confidences.append(float(confidence))
class_ids.append(class_id)

# Non-max suppression
indices = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)

# Draw boxes
if len(indices) > 0:
    for i in indices.flatten():
        x, y, w, h = boxes[i]
        label = f"{classes[class_ids[i]]}: {confidences[i]:.2f}"
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(img, label, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 255, 0), 2)

# Display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(12, 10))
plt.imshow(img_rgb)
plt.axis('off')
plt.show()
```

CEC621 RESIT 2023

QUESTION 1 – Iris Dataset (80% split, Multiple Models)

```

# =====
# RESIT 2023 - QUESTION 1
# Iris Dataset - 80% Training Split
# =====

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split 80% training, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Define models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'SVM': SVC(kernel='rbf', random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

# 1. Train and get scores (5 Marks)
print("="*60)
print("1. MODEL SCORES (80% Training Split)")
print("="*60)
for name, model in models.items():
    model.fit(X_train, y_train)

```

```

score = model.score(X_test, y_test)
print(f"{name:20s} Test Accuracy: {score:.4f}")

# 2. Cross-validation with 6 folds (10 Marks)
print("\n" + "="*60)
print("2. CROSS-VALIDATION SCORES (6-Fold)")
print("="*60)
best_model = None
best_mean_score = 0
best_name = ""

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train, y_train, cv=6, scoring='accuracy')
    mean_score = cv_scores.mean()
    print(f"{name:20s} CV Scores: {cv_scores}")
    print(f"{'':20s} Mean: {mean_score:.4f} (+/- {cv_scores.std()*2:.4f})\n")

    if mean_score > best_mean_score:
        best_mean_score = mean_score
        best_model = model
        best_name = name

print(f"Best model (by CV): {best_name} with mean accuracy {best_mean_score:.4f}")

# 3. Plot confusion matrix for best model (10 Marks)
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
plt.title(f'Confusion Matrix - {best_name} (Iris Dataset)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()

```

QUESTION 2 – Machine Learning vs Deep Learning & MediaPipe

1. Differentiate between Machine Learning and Deep Learning considering independent variables (5 Marks)

Aspect	Machine Learning	Deep Learning
Number of Independent Variables	Works well with fewer variables (tens to hundreds)	Handles thousands to millions of variables (e.g., pixels)
Feature Engineering	Manual feature extraction required	Automatic feature learning from raw data
Data Requirements	Works with smaller datasets	Requires large datasets (millions of samples)
Computational Resources	Moderate (CPU sufficient)	Heavy (GPU/TPU required)
Interpretability	More interpretable (e.g., decision trees, linear models)	Less interpretable (black box)
Performance with High-Dim Data	Degrades with too many variables	Excels with high-dimensional data
Training Time	Faster	Slower
Examples	Linear Regression, SVM, Random Forest	CNNs, RNNs, Transformers

2. Facemesh with MediaPipe (20 Marks)

```

# =====
# RESIT 2023 - QUESTION 2a
# Facemesh with MediaPipe
# =====

import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe FaceMesh
mp_face_mesh = mp.solutions.face_mesh
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

# FaceMesh configuration
face_mesh = mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

def process_facemesh_image(image_path):
    """Apply FaceMesh on a single image"""
    # Read image
    image = cv2.imread(image_path)
    if image is None:
        print("Error: Could not load image")
        return

    # Convert BGR to RGB
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Process
    results = face_mesh.process(image_rgb)

    # Draw landmarks
    annotated_image = image.copy()
    if results.multi_face_landmarks:

```



```

    for face_landmarks in results.multi_face_landmarks:
        mp_drawing.draw_landmarks(
            image=annotated_image,
            landmark_list=face_landmarks,
            connections=mp_face_mesh.FACEMESH_TESSELATION,
            landmark_drawing_spec=None,
            connection_drawing_spec=mp_drawing_styles
                .get_default_face_mesh_tesselation_style()
        )

# Display
plt.figure(figsize=(12, 8))
plt.imshow(cv2.cvtColor(annotated_image, cv2.COLOR_BGR2RGB))
plt.title('FaceMesh Landmarks')
plt.axis('off')
plt.show()

def process_facemesh_webcam():
    """Real-time FaceMesh using webcam"""
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("Error: Could not access webcam")
        return

    print("Press 'q' to quit")

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # Flip horizontally for selfie view
        frame = cv2.flip(frame, 1)

        # Convert to RGB
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(frame_rgb)

```

```

# Draw landmarks
if results.multi_face_landmarks:
    for face_landmarks in results.multi_face_landmarks:
        mp_drawing.draw_landmarks(
            image=frame,
            landmark_list=face_landmarks,
            connections=mp_face_mesh.FACEMESH_TESSELATION,
            landmark_drawing_spec=None,
            connection_drawing_spec=mp_drawing_styles
                .get_default_face_mesh_tesselation_style()
        )

cv2.imshow('FaceMesh', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Pose Detection with MediaPipe:

```

# =====
# RESIT 2023 - QUESTION 2b
# Pose Detection with MediaPipe
# =====

import cv2
import mediapipe as mp

mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles

def pose_detection_webcam():
    """Real-time pose detection using webcam"""
    pose = mp_pose.Pose(
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5
    )

    cap = cv2.VideoCapture(0)

    print("Press 'q' to quit")

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        frame = cv2.flip(frame, 1)
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(frame_rgb)

        if results.pose_landmarks:
            mp_drawing.draw_landmarks(
                frame,
                results.pose_landmarks,
                mp_pose.POSE_CONNECTIONS,
                landmark_drawing_spec=mp_drawing_styles.get_default_pose_landmarks_style()
            )

```

```
cv2.imshow('Pose Detection', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Image Classification with Fashion MNIST:

```

# =====
# RESIT 2023 - QUESTION 2c
# Image Classification with Fashion MNIST
# =====

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# Load Fashion MNIST dataset
fashion_mnist = keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Class labels
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Build CNN model
model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), activation='relu',
                        input_shape=(28, 28, 1)),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, activation='softmax')
])

# Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',

```

```
        metrics=['accuracy'])

# Summary
model.summary()

# Train
history = model.fit(
    X_train.reshape(-1, 28, 28, 1), y_train,
    epochs=10,
    validation_split=0.2,
    batch_size=32
)

# Evaluate
test_loss, test_acc = model.evaluate(X_test.reshape(-1, 28, 28, 1), y_test)
print(f"Test Accuracy: {test_acc:.4f}")

# Predict and visualize
predictions = model.predict(X_test.reshape(-1, 28, 28, 1))
pred_labels = np.argmax(predictions, axis=1)

# Show some results
plt.figure(figsize=(12, 8))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_test[i], cmap='gray')
    plt.title(f'True: {class_names[y_test[i]]}\nPred: {class_names[pred_labels[i]]}')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

CEC621 RESIT 2024

QUESTION 1 – Supervised Learning with Digits Dataset

```

# =====
# RESIT 2024 - QUESTION 1 & 2
# Digits Dataset - Supervised Learning & K-Means
# =====

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
digits = load_digits()
X, y = digits.data, digits.target

# Split (using default 75/25 as per instruction)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# =====
# QUESTION 1: Model Scores (10 Marks)
# =====

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'SVM': SVC(kernel='rbf', random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

print("="*60)
print("QUESTION 1: MODEL SCORES")
print("="*60)

```



```

for name, model in models.items():
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)
    print(f"{name:20s} Test Accuracy: {score:.4f}")

# 2. Cross-validation with 5 folds (10 Marks)
print("\n" + "="*60)
print("QUESTION 1: CROSS-VALIDATION SCORES (5-Fold)")
print("="*60)
best_model = None
best_mean_score = 0
best_name = ""

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    mean_score = cv_scores.mean()
    print(f"{name:20s} CV Scores: {cv_scores}")
    print(f"{'':20s} Mean: {mean_score:.4f} (+/- {cv_scores.std()*2:.4f})\n")

    if mean_score > best_mean_score:
        best_mean_score = mean_score
        best_model = model
        best_name = name

# 3. Plot confusion matrix for best model (10 Marks)
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=digits.target_names,
            yticklabels=digits.target_names)
plt.title(f'Best Model: {best_name} - Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.show()

```

QUESTION 2 – K-Means Clustering (Elbow Method)

```
# =====  
# RESIT 2024 - QUESTION 2  
# K-Means Clustering - Elbow Method (10 Marks)  
# =====  
  
from sklearn.cluster import KMeans  
  
# Use digits dataset (X)  
sse = [] # Sum of Squared Errors  
  
# Try K from 1 to 20  
k_range = range(1, 21)  
for k in k_range:  
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)  
    kmeans.fit(X)  
    sse.append(kmeans.inertia_)  
  
# Plot elbow curve  
plt.figure(figsize=(10, 6))  
plt.plot(k_range, sse, 'bo-', linewidth=2, markersize=8)  
plt.xlabel('Number of Clusters (K)')  
plt.ylabel('Sum of Squared Errors (SSE)')  
plt.title('Elbow Method for Optimal K (Digits Dataset)')  
plt.axvline(x=10, color='red', linestyle='--', label='Optimal K ≈ 10')  
plt.legend()  
plt.grid(True, alpha=0.3)  
plt.show()  
  
# The optimal K is where the elbow occurs  
# For digits dataset (10 classes), optimal K ≈ 10  
print("Optimal K ≈ 10 (matches number of digit classes)")
```

QUESTION 3 – Deep Learning – CIFAR10 with CNN

```

# =====
# RESIT 2024 - QUESTION 3
# CIFAR10 with CNN (30 Marks)
# =====

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Load CIFAR10 dataset
(X_train, y_train), (X_test, y_test) = keras.datasets.cifar10.load_data()

# Normalize pixel values to [0, 1]
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Class names
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

print(f"Training data shape: {X_train.shape}")
print(f"Test data shape: {X_test.shape}")

# =====
# Build CNN: 3 stacks of (Conv + ReLU + Pooling)
# + Fully Connected Output Layer
# =====

model = keras.Sequential([
    # Stack 1
    layers.Conv2D(32, (3, 3), activation='relu', padding='same',
                  input_shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

```

```

# Stack 2
layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
layers.MaxPooling2D((2, 2)),
layers.Dropout(0.3),

# Stack 3
layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
layers.MaxPooling2D((2, 2)),
layers.Dropout(0.4),

# Fully connected output layer
layers.Flatten(),
layers.Dense(256, activation='relu'),
layers.Dropout(0.5),
layers.Dense(10, activation='softmax')
])

```

1. Compile and train it (10 Marks)

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

2. Generate summary of the network (10 Marks)

```

print("\n" + "="*60)
print("MODEL SUMMARY")
print("="*60)
model.summary()

```

Train

```

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=64,
    validation_split=0.2,

```

```

        verbose=1
    )

    # Evaluate
    test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
    print(f"\nTest Accuracy: {test_accuracy:.4f}")

    # 3. Plot confusion matrix (10 Marks)
    y_pred_probs = model.predict(X_test)
    y_pred = np.argmax(y_pred_probs, axis=1)
    y_true = y_test.flatten()

    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names,
                yticklabels=class_names)
    plt.title('Confusion Matrix - CNN Classifier (CIFAR10)')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.tight_layout()
    plt.show()

    # Plot training history
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.title('Loss History')

    plt.subplot(1, 2, 2)
    plt.plot(history.history['accuracy'], label='Training Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.xlabel('Epoch')

```

```
plt.ylabel('Accuracy')
plt.legend()
plt.title('Accuracy History')

plt.tight_layout()
plt.show()
```

CEC621 RESIT 2025

Note: The Resit 2025 paper contains the same MCQs as the main 2025 exam, with the following differences in Section B:

QUESTION 1 – Confusion Matrix (10 Marks)

	Predicted Positive	Predicted Negative
Actual Positive	80	20
Actual Negative	10	90

1. Identify TP, FP, TN, FN (4 Marks)

- **TP = 80** (Actual Positive, Predicted Positive)
- **FP = 10** (Actual Negative, Predicted Positive)
- **TN = 90** (Actual Negative, Predicted Negative)
- **FN = 20** (Actual Positive, Predicted Negative)

2. Formulas and Calculations

a. False Positive Rate (FPR)

$$FPR = \frac{FP}{FP + TN}$$

Answer: FPR = 0.1 (10%)

b. True Positive Rate (TPR)

$$TPR = \frac{TP}{TP + FN}$$

Answer: TPR = 0.8 (80%)

c. F1-Score

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = TPR = 0.8$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Answer: F1-Score = 0.8422 (84.22%)

QUESTION 2 – Digits Dataset (85% split, SVM + Logistic Regression)

The code is identical to Question 2 of the main 2025 exam (above), with the difference being:

- Split: 85% training (15% test)
- Classifiers: SVM and Logistic Regression (instead of SVM and Random Forest)

```
# Same as main exam Question 2, but with Logistic Regression instead of Random Forest
# All code from the main 2025 Question 2 applies here
```

QUESTION 3 – IRIS Dataset with ANN (7 Hidden Neurons)

The code is identical to the main 2025 Question 3, but with **7 neurons** in the hidden layer instead of 6.

1. Sketch ANN with 7 Hidden Neurons (10 Marks)

Same architecture as main exam, but hidden layer has 7 neurons.

2. Parameter Count (2 Marks)

Layer	Weights	Biases	Total
Input → Hidden	$4 \times 7 = 28$	7	35
Hidden → Output	$7 \times 3 = 21$	3	24
Total	49	10	59

Answer: 59 parameters

3. TensorFlow Implementation (28 Marks)

```
# Same as main exam Question 3 code, change Dense(6) to Dense(7)
model = Sequential([
    layers.Dense(7, activation='relu', input_shape=(4,)), name='hidden_layer'),
    layers.Dense(3, activation='softmax', name='output_layer')
])
# Rest of code identical
```

Study Notes – Key Concepts Summary

Supervised Learning

- **Classification:** Logistic Regression, SVM, Decision Trees, Random Forest, Neural Networks

- **Regression:** Linear Regression, Ridge, Lasso
- **Metrics:** Accuracy, Precision, Recall, F1-Score, Confusion Matrix

Unsupervised Learning

- **Clustering:** K-Means, DBSCAN, Hierarchical
- **Dimensionality Reduction:** PCA, t-SNE

Deep Learning

- **ANN:** Fully connected layers, used for tabular data
- **CNN:** Convolutional layers, used for images
- **RNN/LSTM:** Used for sequential data (NLP, time series)

Key Libraries

- **Scikit-learn:** Traditional ML (models, preprocessing, metrics)
- **TensorFlow/Keras:** Deep learning
- **PyTorch:** Deep learning
- **MediaPipe:** Computer vision pipelines

Model Evaluation

- **Train/Test Split:** Assess generalization
- **Cross-Validation:** Robust performance estimate
- **Confusion Matrix:** Visualize classification performance
- **ROC/AUC:** Assess binary classification

End of CEC621 Solutions